

Document number:	P2548R3
Date:	2022-11-11
Project:	Programming Language C++
Audience:	LEWG
Reply-to:	Michael Florian Hava ¹ < mfh.cpp@gmail.com >

copyable_function

Abstract

This paper proposes a replacement for function in the form of a copyable variant of move_only_function.

Tony Table

Before		Proposed	
<code>auto lambda{[&]() /*const*/ { ... };</code>		<code>auto lambda{[&]() /*const*/ { ... };</code>	
<code>function<void(void)> func{lambda};</code>	✓	<code>copyable_function<void(void)> func0{lambda};</code>	✓
<code>const auto & ref{func};</code>		<code>const auto & ref0{func0};</code>	
<code>func();</code>	✓	<code>func0();</code>	✓
<code>ref();</code>	✓	<code>ref0(); //operator() is NOT const!</code>	✗
		<code>copyable_function<void(void) const> func1{lambda};</code>	✓
		<code>const auto & ref1{func1};</code>	
		<code>func1();</code>	✓
		<code>ref1(); //operator() is const!</code>	✓
<code>auto lambda{[&]() mutable { ... };</code>		<code>auto lambda{[&]() mutable { ... };</code>	
<code>function<void(void)> func{lambda};</code>	✓	<code>copyable_function<void(void)> func{lambda};</code>	✓
<code>const auto & ref{func};</code>		<code>const auto & ref{func};</code>	
<code>func();</code>	✓	<code>func();</code>	✓
<code>ref(); //operator() is const!</code>	⚠	<code>ref(); //operator() is NOT const!</code>	✗
<code>//this is the infamous constness-bug</code>	✓		
		<code>copyable_function<void(void) const> tmp{lambda};</code>	✗

Revisions

R0: Initial version

R1:

- Incorporated the changes proposed for move_only_function in [\[P2511R2\]](#).
- Added wording for conversions from copyable_function to move_only_function.

R2:

- Removed changes adopted from [\[P2511R2\]](#) as that proposal didn't reach consensus in the 2022-10 LEWG electronic polling.

R3: Updates after LEWG Review on 2022-11-08:

- Fixed requirements on callables in the design section – copy-construct-ability is sufficient.
- Removed open question on the deprecation of function.
- Replaced previously proposed conversion operators to move_only_function.

¹ RISC Software GmbH, Softwarepark 32a, 4232 Hagenberg, Austria, michael.hava@risc-software.at

- Added section on conversions between standard library polymorphic function wrappers.
- Added section on potential allocator support.

Motivation

C++11 added `function`, a type-erased function wrapper that can represent any *copyable* callable matching the function signature `R(Args...)`. Since its introduction, there have been identified several issues – including the infamous constness-bug – with its design (see [\[N4159\]](#)).

[\[P0288R9\]](#) introduced `move_only_function`, a *move-only* type-erased callable wrapper. In addition to dropping the *copyable* requirement, `move_only_function` extends the supported signature to `R(Args...) constop (&|&&)op noexceptop` and forwards all qualifiers to its call operator, introduces a strong non-empty precondition for invocation instead of throwing `bad_function_call` and drops the dependency to typeid/RTTI (there is no equivalent to function's `target_type()` or `target()`).

Concurrently, [\[P0792R10\]](#) introduced `function_ref`, a type-erased non-owning reference to any callable matching a function signature in the form of `R(Args...) constop noexceptop`. Like `move_only_function`, it forwards the `noexcept`-qualifier to its call operator. As `function_ref` acts like a reference, it does not support `ref`-qualifiers and does not forward the `const`-qualifier to its call operator.

As a result, `function` is now the only type-erased function wrapper not supporting any form of qualifiers in its signature. Whilst amending `function` with support for `ref`/`noexcept`-qualifiers would be a straightforward extension, the same is not true for the `const`-qualifier due to the long-standing constness-bug. Without proper support for the `const`-qualifier, `function` would still be inconsistent with its closest relative.

Therefore, this paper proposes to introduce a replacement to `function` in the form of `copyable_function`, a class that closely mirrors the design of `move_only_function` and adds *copyability* as an additional affordance.

Design space

The main goal of this paper is consistency between the *move-only* and *copyable* type-erased function wrappers. Therefore, we follow the design of `move_only_function` very closely and only introduce three extensions:

1. Adding a copy constructor
2. Adding a copy assignment operator
3. Requiring callables to be copy-constructible

Conversions between function wrappers

Given the proliferation of proposals for polymorphic function wrappers, LEWG requested an evaluation of the „conversion story“ of these types.

		To			
		function	move_only_function	copyable_function	function_ref
From	function		✓	✓	✓
	move_only_function	✗		✗	✓
	copyable_function	✓	✓		✓
	function_ref	✓	✓	✓	

It is recommended that implementors do not perform additional allocations when converting from a `copyable_function` instantiation to a compatible `move_only_function` instantiation, but this is left as quality-of-implementation.

Concerning allocator support

After having reviewed R2, LEWG requested a statement about potential allocator support. As this proposal aims for feature parity with `move_only_function` (apart from the extensions mentioned above) and considering the somewhat recent removal of allocator support from function [\[P0302\]](#), we refrain from adding allocator support to `copyable_function`. We welcome an independent paper introducing said support to both classes.

Impact on the Standard

This proposal is a pure library addition. It introduces a new class template and adds an optimization requirement to an existing class template.

Implementation Experience

The proposed design has been implemented at <https://github.com/MFHava/P2548>.

Proposed Wording

Wording is relative to [\[N4910\]](#). Additions are presented like **this**, removals like **this**.

[\[version.syn\]](#)

In [\[version.syn\]](#), add:

```
#define cpp lib copyable function YYYYMMML //also in <functional>
```

Adjust the placeholder value as needed to denote this proposal's date of adoption.

[\[functional.syn\]](#)

In [\[functional.syn\]](#), in the synopsis, add the proposed class template:

```
// 22.10.17.4, move only wrapper
template<class... S> class move_only_function; // not defined
template<class R, class... ArgTypes>
    class move_only_function<R(ArgTypes...) cv ref noexcept(noex)>; // see below

// 22.10.17.5, copyable wrapper
template<class... S> class copyable_function; // not defined
template<class R, class... ArgTypes>
    class copyable_function<R(ArgTypes...) cv ref noexcept(noex)>; // see below

// 22.10.18, searchers
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    class default_searcher;
```

[\[func.wrap\]](#)

In [\[func.wrap\]](#), insert the following section at the end of **Polymorphic function wrappers**:

	22.10.17.5 Copyable wrapper	[func.wrap.copy]
	22.10.17.5.1 General	[func.wrap.copy.general]
1	The header provides partial specializations of <code>copyable_function</code> for each combination of the possible replacements of the placeholders <code>cv</code> , <code>ref</code> , and <code>noex</code> where	
1.1	— <code>cv</code> is either <code>const</code> or empty,	
1.2	— <code>ref</code> is either <code>&</code> , <code>&&</code> , or empty, and	
1.3	— <code>noex</code> is either <code>true</code> or <code>false</code> .	
2	For each of the possible combinations of the placeholders mentioned above, there is a placeholder <code>inv-quals</code> defined as follows:	
2.1	— If <code>ref</code> is empty, let <code>inv-quals</code> be <code>cv&</code> ,	
2.2	— otherwise, let <code>inv-quals</code> be <code>cv ref</code> .	

22.10.17.5.2 Class template copyable function

[func.wrap.copy.class]

```
namespace std {
    template<class... S> class copyable function; // not defined

    template<class R, class... ArgTypes>
    class copyable function<R(ArgTypes...) cv ref noexcept(noex)> {
    public:
        using result type = R;

        // 22.10.17.5.3, constructors, assignments, and destructors
        copyable function() noexcept;
        copyable function(nullptr t) noexcept;
        copyable function(const copyable function&);
        copyable function(copyable function&&) noexcept;
        template<class F> copyable function(F&&);
        template<class T, class... Args>
            explicit copyable function(in place type t<T>, Args&&...);
        template<class T, class U, class... Args>
            explicit copyable function(in place type t<T>, initializer list<U>, Args&&...);

        copyable function& operator=(const copyable function&);
        copyable function& operator=(copyable function&&);
        copyable function& operator=(nullptr t) noexcept;
        template<class F> copyable function& operator=(F&&);

        ~copyable function();

        // 22.10.17.5.4, invocation
        explicit operator bool() const noexcept;
        R operator()(ArgTypes...) cv ref noexcept(noex);

        // 22.10.17.5.5, utility
        void swap(copyable function&) noexcept;
        friend void swap(copyable function&, copyable function&) noexcept;
        friend bool operator==(const copyable function&, nullptr t) noexcept;

    private:
        template<class VT>
            static constexpr bool is-callable-from = see below; //exposition only
    };
}
```

1 The copyable function class template provides polymorphic wrappers that generalize the notion of a callable object (22.10.3). These wrappers can store, copy, move, and call arbitrary callable objects, given a call signature. Within this subclause, *call-args* is an argument pack with elements that have types *ArgTypes&&...* respectively.

2 **Recommended practice:** Implementations should avoid the use of dynamically allocated memory for a small contained value.

[Note 1: Such small-object optimization can only be applied to a type *T* for which *is_nothrow_constructible_v<T>* is true. — end note]

22.10.17.5.3 Constructors, assignment, and destructor

[func.wrap.copy.ctor]

```
template<class VT>
    static constexpr bool is-callable-from = see below;

    If noex is true, is-callable-from<VT> is equal to:
        is_nothrow_invocable_r v<R, VT cv ref, ArgTypes...> &&
        is_nothrow_invocable_r v<R, VT inv-quals, ArgTypes...>
    Otherwise, is-callable-from<VT> is equal to:
        is_invocable_r v<R, VT cv ref, ArgTypes...> &&
        is_invocable_r v<R, VT inv-quals, ArgTypes...>
```

```
copyable function() noexcept;
copyable function(nullptr t) noexcept;
    Postconditions: *this has no target object;
```

```
copyable function(const copyable function& f)
    Postconditions: *this has no target object if f had no target object
    Otherwise, the target object of *this is a copy of the target object of f.
    Throws: Any exception thrown by the initialization of the target object. May throw bad_alloc.
```

```
copyable function(copyable function&& f) noexcept;
    Postconditions: The target object of *this is the target object f had before construction, and f is in a valid state with an unspecified value.
```

```
template<class F> copyable function(F&& f);
    Let VT be decay t<F>.
```

Constraints:

- 7.1 — remove_cvref t<F> is not the same as copyable function, and
- 7.2 — remove_cvref t<F> is not a specialization of in place type t, and
- 7.3 — is-callable-from<VT> is true.

Mandates:

- 8.1 — is_constructible v<VT, F> is true, and
- 8.2 — is_copy_constructible v<VT> is true.

Preconditions: VT meets the Cpp17Destructible requirements, and if is_move_constructible v<VT> is true, VT meets the Cpp17MoveConstructible requirements.

Postconditions: **this* has no target object if any of the following hold:
 — *f* is a null function pointer value, or
 — *f* is a null member function pointer value, or
 — remove_cvref_t<*F*> is a specialization of the copyable function class template, and *f* has no target object.
 Otherwise, **this* has a target object of type VT direct-non-list-initialized with std::forward<*F*>(*f*).
Throws: Any exception thrown by the initialization of the target object. May throw bad_alloc unless VT is a function pointer or a specialization of reference_wrapper.

```

template<class T, class... Args>
explicit copyable_function(in_place_type_t<T>, Args&&... args);
  Let VT be decay_t<T>.
  Constraints:
    — is_constructible_v<VT, Args...> is true, and
    — is_callable_from<VT> is true.
  Mandates:
    — VT is the same type as T, and
    — is_copy_constructible_v<VT> is true.
  Preconditions: VT meets the Cpp17Destructible requirements, and if is_move_constructible_v<VT> is true, VT meets the Cpp17MoveConstructible requirements.
  Postconditions: *this has a target object d of type VT direct-non-list-initialized with std::forward<Args>(args)...
  Throws: Any exception thrown by the initialization of the target object. May throw bad_alloc unless VT is a pointer or a specialization of reference_wrapper.
  
```

```

template<class T, class U, class... Args>
explicit copyable_function(in_place_type_t<T>, initializer_list<U> ilist, Args&&... args);
  Let VT be decay_t<T>.
  Constraints:
    — is_constructible_v<VT, initializer_list<U>&, Args...> is true, and
    — is_callable_from<VT> is true.
  Mandates:
    — VT is the same type as T, and
    — is_copy_constructible_v<VT> is true.
  Preconditions: VT meets the Cpp17Destructible requirements, and if is_move_constructible_v<VT> is true, VT meets the Cpp17MoveConstructible requirements.
  Postconditions: *this has a target object d of type VT direct-non-list-initialized with ilist, std::forward<Args>(args)...
  Throws: Any exception thrown by the initialization of the target object. May throw bad_alloc unless VT is a pointer or a specialization of reference_wrapper.
  
```

```

copyable_function& operator=(const copyable_function& f);
  Effects: Equivalent to: copyable_function(f).swap(*this);
  Returns: *this.

copyable_function& operator=(copyable_function&& f);
  Effects: Equivalent to: copyable_function(std::move(f)).swap(*this);
  Returns: *this.

copyable_function& operator=(nullptr_t) noexcept;
  Effects: Destroys the target object of *this, if any.
  Returns: *this.

template<class F> copyable_function& operator=(F&& f);
  Effects: Equivalent to: copyable_function(std::forward<F>(f)).swap(*this);
  Returns: *this.

~copyable_function();
  Effects: Destroys the target object of *this, if any.
  
```

22.10.17.5.4 Invocation [func.wrap.copy.inv]
 explicit operator bool() const noexcept;
 Returns: true if **this* has a target object, otherwise false.

```

R operator()(ArgTypes... args) cv_ref noexcept(noex);
  Preconditions: *this has a target object.
  Effects: Equivalent to:
    return INVOKE<R>(static_cast<F inv_qual>(f), std::forward<ArgTypes>(args)...);
  where f is an lvalue designating the target object of *this and F is the type of f.
  
```

22.10.17.5.5 Utility [func.wrap.copy.util]
 void swap(copyable_function& other) noexcept;
 Effects: Exchanges the target objects of **this* and *other*.

```

friend void swap(copyable_function& f1, copyable_function& f2) noexcept;
  Effects: Equivalent to f1.swap(f2).

friend bool operator==(const copyable_function& f, nullptr_t) noexcept;
  
```

[func.wrap.move.ctor]

In [func.wrap.move.ctor], insert the following:

```

4   template<class F> move_only_function(F&& f);
   Let VT be decay_t<F>.
5   Constraints:
(5.1) — remove_cvref_t<F> is not the same as move_only_function, and
(5.2) — remove_cvref_t<F> is not a specialization of in_place_type_t, and
(5.3) — is_callable_from<VT> is true.
6   Mandates: is_constructible_v<VT, F> is true.
7   Preconditions: VT meets the Cpp17Destructible requirements, and if is_move_constructible_v<VT> is true, VT meets the
   Cpp17MoveConstructible requirements.
8   Postconditions: *this has no target object if any of the following hold:
(8.1) — f is a null function pointer value, or
(8.2) — f is a null member function pointer value, or
(8.3) — remove_cvref_t<F> is a specialization of the move_only_function class template, and f has no target object.
   Otherwise, *this has a target object of type VT direct-non-list-initialized with std::forward<F>(f).
9   Throws: Any exception thrown by the initialization of the target object. May throw bad_alloc unless VT is a function pointer
   or a specialization of reference_wrapper, or F is copyable_function<R(Args...) cv ref noexcept(noex)>.

```

Acknowledgements

Thanks to [RISC Software GmbH](#) for supporting this work. Thanks to Peter Kulczycki for proof reading and discussions. Thanks to Matt Calabrese for helping to get conversions to move_only_function to work.